Dental Clinic Staff Management App: Product Brief and Feature List

Part 1: Product Brief

Overview

A web app that helps a dental clinic with two locations manage its staff in one place: who works when, who is on leave, and whether required credentials are up to date. It replaces paper rosters, spreadsheets, and scattered chat messages.

Problem

Clinic managers typically build schedules by hand, collect leave requests through WhatsApp or email, and track licence expiry dates on paper. This causes double bookings, understaffed days, missed renewals, and constant back-and-forth messaging.

Target users

- **Clinic Owner / Admin:** sets up the clinic, manages roles, sees both locations and everyone's pay.
- **Practice Manager:** runs one location: builds schedules, approves leave, monitors credentials, and corrects attendance records.
- **Clinical staff (dentists, hygienists, dental assistants):** view their own shifts, clock in and out, request leave, update availability.
- **Reception staff:** view the daily roster to know who is working and who is on leave.

Goals

1. Give managers a clear weekly view of who is working, where, and when.
2. Make leave requests and approvals fast, with no chasing by message.
3. Prevent expired licences and certifications from going unnoticed.
4. Let every staff member see their own schedule and clock in and out from a phone.

Scope for version 1

- Staff profiles, roles, and two locations
- Shift scheduling across both locations
- Schedule checks: conflicts, minimum staffing, expired licences, and overtime warnings
- Leave requests and approvals
- Clock in and out with attendance records
- Credential and licence expiry tracking
- Clinic settings and policies (leave, breaks, overtime, closed days, coverage rules)
- Pay and compensation records (Owner / Admin only)
- Notifications
- Basic reports

Out of scope for version 1

- Patient records, appointments, or billing
- Payroll calculation and payments (pay data is stored, but pay is not calculated here)
- More than two locations, or franchise management
- Native mobile apps (the web app should be mobile-friendly instead)
- Integrations with practice management or accounting software

Success criteria

- A manager can build and publish a full weekly schedule in under 15 minutes.
- A leave request can be submitted and decided without any messages outside the app.
- Staff are warned at least 30 days before a credential expires.
- Every staff member can find their next shift in under 10 seconds.
- Every schedule warning (conflict, minimum staffing, expired licence, overtime) appears before publishing, and an expired required licence cannot be scheduled without a logged override.

Assumptions and open questions

- Assumed: one clinic location with 5 to 30 staff. Does the target clinic have multiple locations? Answer: 2 locations, 15 staffs per location.
- Assumed: shifts are assigned per person per day, not per chair or room. Do they need chair or room assignment? Answer: Per person per day.
- Assumed: pay information is not stored in version 1. Is any compensation data required? Answer: Pay information and compensation data need to store.
- Open: do staff clock in and out in the app, or only view scheduled shifts? Answer: clock in and out in the app.

Part 2: Feature List

Priority key: **Must** is required for version 1, **Should** is valuable but can follow soon after, **Later** is planned for a future version.

Module 1: Authentication and roles

- **Must:** As a staff member, I want to log in securely with email and password so that only I can access my account.
- **Must:** As an admin, I want to invite staff by email so that they can set up their own accounts.
- **Must:** As an admin, I want to assign each person a role and a location so that they only see what their role and location allow.
- **Should:** As a user, I want to reset my own password so that I do not need to contact an admin.
- **Later:** As an admin, I want to turn on two-factor login for extra security.

Module 2: Staff profiles

- **Must:** As a manager, I want to add and edit staff profiles (name, role, home location, contact details, start date) so that records are accurate.
- **Must:** As a manager, I want to see a searchable staff directory so that I can find anyone quickly.

- **Should:** As a manager, I want to deactivate a staff member who has left without deleting their history.
- **Should:** As a staff member, I want to update my own contact details.
- **Later:** As a manager, I want to attach documents (contracts, certificates) to a profile.

Module 3: Shift scheduling

- **Must:** As a manager, I want a weekly calendar showing every staff member's shifts so that I can see coverage at a glance.
- **Must:** As a manager, I want to create, edit, and delete shifts so that I can build the schedule.
- **Must:** As a staff member, I want to see my own upcoming shifts at either location, including on my phone.
- **Must:** As a manager, I want to schedule a staff member from the other location at my location so that I can cover gaps.
- **Must:** As a manager, I want a warning when I schedule someone who is on approved leave or already has an overlapping shift at either location.
- **Must:** As a manager, I want a warning when a day at my location falls below the minimum staffing for a job title, so that I can fix gaps before publishing.
- **Must:** As a manager, I want a warning when I schedule someone whose required licence or certification is expired on the shift date, with publishing blocked unless an Owner / Admin overrides it, so that no one works without a valid credential.
- **Must:** As a manager, I want each person's weekly hours across both locations shown on the schedule, with a warning above the overtime threshold (40 hours by default) for overtime-eligible staff.
- **Should:** As a manager, I want closed days shown on the calendar so that I do not schedule shifts on them.
- **Should:** As a manager, I want to copy last week's schedule so that I do not rebuild it from scratch.
- **Should:** As a manager, I want to publish a schedule so that staff only see it once it is final.
- **Later:** As a manager, I want to define shift templates (morning, afternoon, full day).
- **Later:** As staff, I want to request a shift swap with a colleague.

Module 4: Leave management

- **Must:** As a staff member, I want to request leave by choosing dates and a type (annual, sick, other), including half days, so that my manager can review it.
- **Must:** As a manager, I want to approve or reject requests, with an optional comment, so that decisions are recorded.
- **Must:** As a staff member, I want to see the status of my requests.
- **Must:** As an Owner / Admin, I want my own leave recorded as approved automatically, or approved by a second Owner / Admin if there is one, so that every absence is on the calendar and counted in staffing checks.
- **Should:** As a manager, I want a team leave calendar so that I can see overlaps before approving.
- **Should:** As a manager, I want to see whether a request would leave my location below minimum staffing before I decide.

- **Should:** As a staff member, I want to see my remaining leave balance out of my yearly allowance (15 days by default).
- **Should:** As a staff member, I want days the clinic is closed not to be deducted from my leave, so that my balance is fair.
- **Should:** As an Owner / Admin, I want to set the yearly allowance (15 days by default), the leave year, the carry-over cap, pro-rating for new hires, which leave types count against it, and any separate sick allowance, so that the app matches the clinic's policy and its state's sick leave rules.

Module 5: Credentials and licence tracking

- **Must:** As a manager, I want to record each staff member's licences and certifications with expiry dates so that nothing lapses.
- **Must:** As a manager, I want a list of credentials expiring soon so that I can act early.
- **Should:** As a staff member, I want to see my own credentials and upcoming expiry dates.
- **Later:** As a manager, I want to upload a copy of each certificate.

Module 6: Notifications

- **Must:** As a staff member, I want to be notified when my schedule is published or changed.
- **Must:** As a manager, I want to be notified of new leave requests.
- **Must:** As a staff member, I want to be told when my leave request is decided.
- **Should:** As a manager and staff member, I want expiry reminders 30 days before a credential lapses.
- **Later:** As a user, I want to choose between email and in-app notifications.

Module 7: Reports

- **Should:** As a manager, I want a report per person per week or month showing hours scheduled, hours worked, unpaid breaks, paid hours, and overtime hours.
- **Should:** As a manager, I want a summary of leave taken by staff member.
- **Later:** As an owner, I want to export reports to CSV or PDF.

Module 8: Attendance (clock in and out)

- **Must:** As a staff member, I want to clock in and out in the app, choosing my location (it defaults to today's shift location, otherwise my home location), so that my working hours are recorded at the right place.
- **Must:** As a staff member, I want to see my own attendance records so that I can check my hours.
- **Must:** As a manager, I want to see attendance records for my location so that I know who worked and for how long.
- **Must:** As a manager, I want a clock-in without a scheduled shift flagged as unscheduled so that I can confirm it, while the time is still recorded.
- **Must:** As a manager, I want missing clock-outs flagged so that I can correct them, with each correction logged.
- **Must:** As a manager, I want unpaid break time deducted from each attendance record by default, and to correct it, with the change logged, when someone worked through their break.
- **Must:** As a manager, I want attendance records flagged when a person's actual weekly hours across both locations exceed the overtime threshold.

- **Later:** As an Owner / Admin, I want to export attendance hours to CSV.

Module 9: Pay and compensation

- **Must:** As an Owner / Admin, I want to record and edit each staff member's pay and compensation so that records are complete.
- **Must:** As a staff member, I want to see my own pay and compensation details.
- **Must:** As an Owner / Admin, I want pay data hidden from managers and other staff so that it stays confidential.
- **Should:** As an Owner / Admin, I want a history of pay changes so that I can see what changed and when.

Module 10: Clinic settings and policies

- **Must:** As an Owner / Admin, I want to set the minimum daily headcount for each job title, per location and weekday, so that the app can warn about understaffed days.
- **Must:** As an Owner / Admin, I want to define which licences and certifications each job title requires, and whether an expired one blocks scheduling.
- **Must:** As an Owner / Admin, I want to mark each person as overtime-eligible or not, and set the workweek start day and the weekly overtime threshold (40 hours by default).
- **Must:** As an Owner / Admin, I want to set the default unpaid break length and override it on a shift.
- **Should:** As an Owner / Admin, I want a closed-day calendar per location, prefilled with US federal holidays, so that closures are excluded from schedules, staffing checks, and leave deductions.
- **Later:** As an Owner / Admin, I want to set coverage by time of day, for example a dentist present during all open hours.

Cross-cutting requirements

- **Must:** Fully usable on a mobile browser, since many staff will check shifts and clock in on their phones.
- **Must:** Role-based access, so staff see only their own data and managers see only their own location, unless their role allows more.
- **Must:** Secure handling of personal data, including pay data, with HTTPS and encrypted storage.
- **Must:** An audit log of changes to schedules, leave decisions, attendance corrections, pay, and roles.

Part 3: Roles and Permissions Matrix

Four roles cover the clinic. Everyone sees their own data, managers are limited to their own location, and only the Owner / Admin sees both locations and everyone's pay.

Roles: **Owner / Admin** runs the whole organisation. **Practice Manager** runs one location (one per location). **Clinical staff** means dentists, hygienists, and dental assistants. **Reception** covers front-desk staff.

Action	Owner / Admin	Practice Manager	Clinical staff	Reception
Invite staff and assign roles	Both locations	No	No	No
View staff directory (name, role, location)	Both locations	Both locations, names and roles only	Own location	Own location
View full staff profile (contact details, start date)	Both locations	Own location	Own only	Own only
Add, edit, or deactivate staff	Both locations	Own location	No	No
View weekly schedule	Both locations	Own location	Own location, published only; own shifts at either location	Own location, published only; own shifts at either location
Create, edit, and publish shifts	Both locations	Own location, for staff from either location	No	No
Request leave	Own only	Own only	Own only	Own only
Approve or reject leave	Both locations; own leave is recorded as approved	Own location, not own leave	No	No
View team leave calendar	Both locations	Own location	Own location, dates only	Own location, dates only
Clock in and out	Own only	Own only	Own only	Own only
View attendance records	Both locations	Own location	Own only	Own only
Correct attendance records (missing clock-outs, unscheduled clock-ins, breaks)	Both locations	Own location, not own records	No	No

Action	Owner / Admin	Practice Manager	Clinical staff	Reception
Manage credentials and licences	Both locations	Own location	View own	View own
View pay and compensation	Both locations	Own only	Own only	Own only
Edit pay and compensation	Both locations	No	No	No
View reports (hours, leave)	Both locations	Own location	No	No
View audit log	Both locations	No	No	No
Manage clinic settings and policies (coverage rules, credential requirements, overtime, breaks, closed days, leave policy)	Both locations	No	No	No
Override a licence block when publishing a schedule	Both locations	No	No	No

Rules behind the matrix

- Everyone can always see their own schedule, leave, attendance, credentials, and pay.
- Pay and compensation is the most restricted data: only the Owner / Admin can see or change anyone else's.
- Managers cannot approve their own leave or correct their own attendance; those go to the Owner / Admin.
- Every schedule change, leave decision, attendance correction, pay edit, and role change is written to the audit log.
- Each person has one home location. "Own location" in the table means staff whose home is that location, except for shifts and attendance, which follow the location where the shift takes place.
- Staff can be scheduled at either location. The manager of the location where the shift happens creates it, and the conflict check looks across both locations so nobody is double-booked.
- Each location has its own Practice Manager. Reception can see both who is working and who is on leave.

- Only the Owner / Admin can change clinic settings and policies (leave policy, coverage rules, credential requirements, overtime, breaks, closed days) and override a licence block when publishing. Every change and override is written to the audit log.
- The Owner / Admin's own leave is recorded as approved automatically, still appears on the calendar, and counts in staffing checks. If the clinic has a second Owner / Admin, that person approves it instead.

Open questions

- Open: do staff ever work shifts at both locations? If yes, scheduling and visibility need a cross-location rule. Answer: Yes.
- Open: does each location have its own Practice Manager, or one manager for both? Answer: Each location have its own Practice Manager.
- Open: should Reception see only who is working, or also who is on leave? Answer: Yes. Should be able to see both.

Part 4: User Flows

Five flows cover the Must features. Each one names who starts it, the steps in order, and what happens when something goes wrong.

Flow 1: Invite and onboard staff

Started by the Owner / Admin. A new staff member goes from invitation to their first schedule view in five steps.

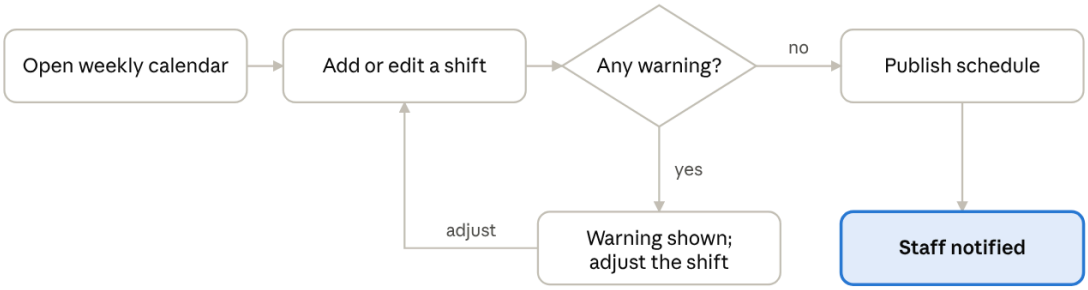
1. The Owner / Admin opens the staff list and chooses Invite staff.
2. They enter the person's name, work email, role, and location.
3. The app emails an invitation with a secure link.
4. The staff member opens the link and sets a password.
5. They confirm their contact details and land on their own schedule.

If the link is not used or has expired, the Owner / Admin resends it from the staff list. If the email already belongs to a profile, the app opens that profile instead of creating a duplicate.

Flow 2: Build and publish the weekly schedule

Started by the Practice Manager.

Every schedule is checked before it is published



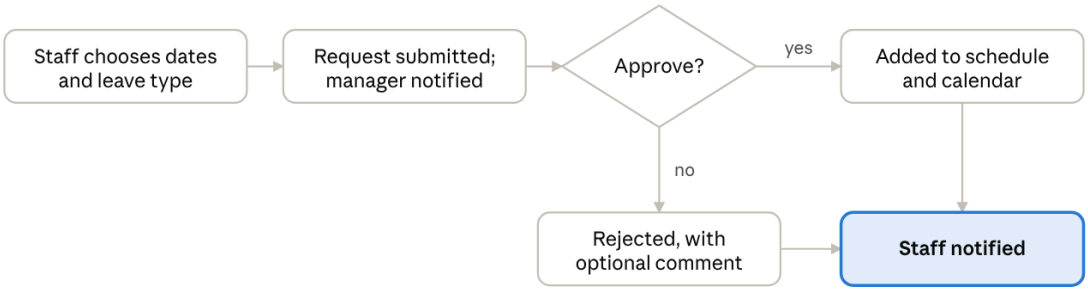
schedule flow · 5 steps, 1 decision

A shift that clashes with approved leave or another shift, at either location, raises a warning, and the manager adjusts it before moving on. Before publishing, the app also checks three more things: days below minimum staffing, expired required licences (which block publishing unless the Owner / Admin overrides it), and weekly hours above the overtime threshold. Staff are notified once the schedule is published.

Flow 3: Request and decide leave

Started by any staff member. Their Practice Manager decides, or the Owner / Admin for a manager's own leave. The Owner / Admin's own leave is recorded as approved automatically, or approved by a second Owner / Admin if there is one.

Every leave request ends with a recorded decision and a notification

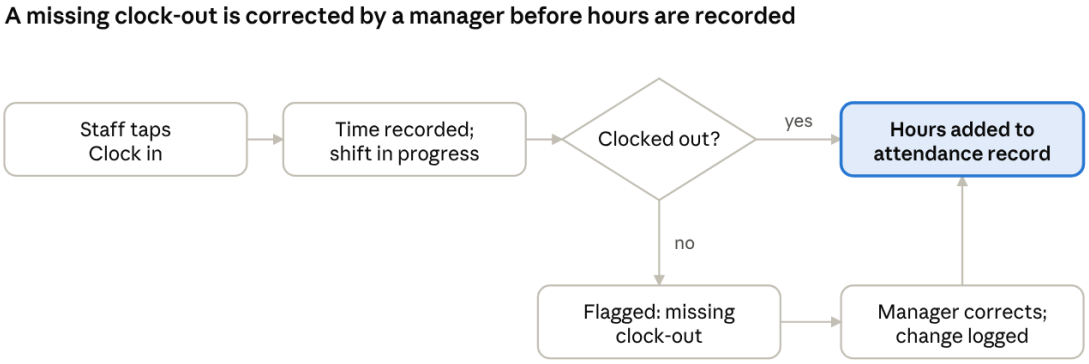


leave flow · 5 steps, 1 decision

Before deciding, the manager sees whether the days off would leave a location below minimum staffing. Approved leave is added to the schedule and team calendar. Either way, the staff member is notified of the decision.

Flow 4: Clock in and out

Started by any staff member at the start of a shift. They confirm their location, which defaults to today's shift location, otherwise their home location.



clock-in flow · 5 steps, 1 decision

If a staff member forgets to clock out, the record is flagged and a manager corrects it. The correction is written to the audit log. A clock-in with no scheduled shift is still recorded, flagged as unscheduled, and sent to that location's manager to confirm. An unpaid break is deducted from each record by default, and a manager can correct it, with a logged change, when someone worked through their break.

Flow 5: Credential expiry reminders

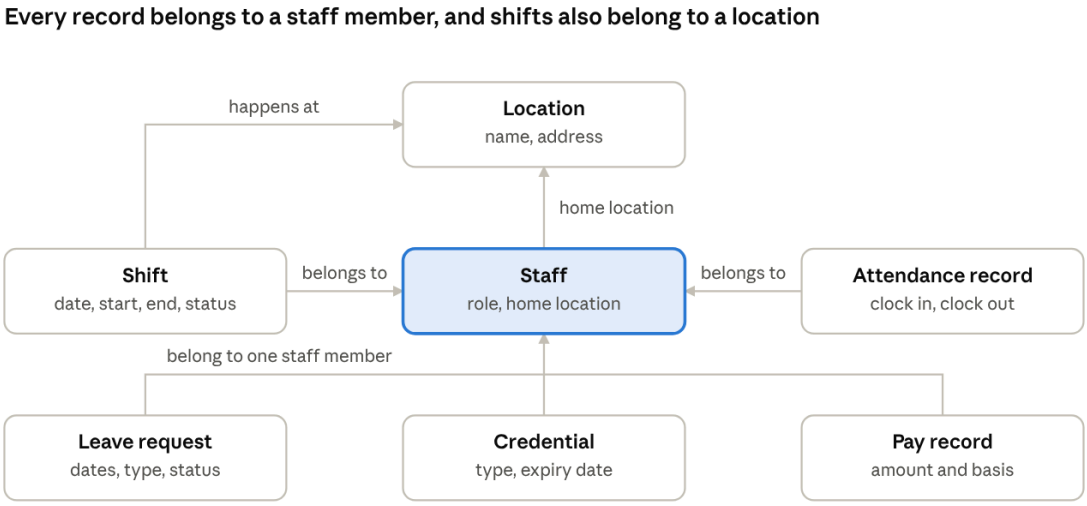
Started by the Practice Manager or Owner / Admin. A licence is recorded once and then watched until it is renewed.

1. The manager adds a credential to a staff profile: type and expiry date.
2. The credential appears in the Credentials list and on the person's own profile.
3. The expiring-soon list shows credentials that are close to their expiry date.
4. Thirty days before expiry, the manager and the staff member get a reminder.
5. After renewal, the manager updates the expiry date and the reminder clears.

If a credential passes its expiry date without an update, it shows as expired in the list and on the profile until the manager fixes it. If the credential is required for the person's job title, it also blocks publishing any schedule that includes them after the expiry date, unless the Owner / Admin overrides it.

Part 5: Data Model

Sixteen entities hold everything the app needs. Almost every record belongs to one staff member, and both staff and shifts belong to a location. These fields are a proposed first version to confirm before building.



data model · 7 entities, arrows point to the parent

Arrows point from a record to the one it belongs to. Invitations, notifications, audit log entries, and the settings and policy entities (leave policy, clinic setting, coverage rule, credential type, closed day, and job title) are left off the diagram to keep it readable.

Entity	What it stores	Key fields
Location	One clinic site	name, address
Job title	The fixed list of job titles	name, role it belongs to (for example, Hygienist belongs to Clinical staff)
Staff	A person who works at the clinic and signs in	name, work email, role, job title, home location, phone, start date, status (active or deactivated), overtime-eligible (yes or no), employment type (full-time or part-time)
Invitation	A pending invitation to join	email, role, job title, home location, secure token, expires, status (pending, accepted, expired)
Shift	One person working one day at one location	staff, location, date, start time, end time, unpaid break minutes, status (draft or published)
Leave request	A request for time off	staff, start date, end date, half-day flags for the first and

Entity	What it stores	Key fields
		last day, days counted, type (annual, sick, other), status (pending, approved, rejected), decided by, decision comment
Leave policy	The clinic's leave rules	yearly allowance (15 days by default), leave year start (calendar year by default), carry-over cap, pro-rate new hires (yes or no), leave types that count, separate sick allowance (optional)
Attendance record	Actual clock-in and clock-out times	staff, shift (optional), location, unscheduled (yes or no), clock in, clock out, unpaid break minutes, flagged (missing clock-out), corrected by, correction note
Credential type	A kind of licence or certification	name, job titles that require it, blocks scheduling when expired (yes or no)
Credential	A licence or certification held by one person	staff, credential type, expiry date
Coverage rule	Minimum staffing for one job title	location, job title, weekday, minimum headcount
Closed day	A holiday or closure	date, location (or both), name
Clinic setting	Clinic-wide values	time zone, workweek start day, weekly overtime threshold (40 hours by default), default unpaid break minutes
Pay record	Pay and compensation, visible to others only for the Owner / Admin	staff, amount, basis (hourly or salary), effective from
Notification	A message sent to one person	recipient, type, message, read, created at
Audit log entry	A record of who changed what	who, action, record changed, old value, new value, when

Key rules

- Role has four values: Owner / Admin, Practice Manager, Clinical staff, and Reception. Job title (dentist, hygienist, dental assistant, receptionist) is a separate field chosen from a fixed list, so all clinical staff share one set of permissions.
- A shift has its own location, which can differ from the person's home location. This is what allows scheduling at either location.
- A leave request is decided by the manager of the person's home location, or by the Owner / Admin for a manager's own request.
- An attendance record has its own location, chosen at clock-in and defaulting to the shift's location. The link to a shift is optional, so a clock-in without a scheduled shift is still recorded and flagged as unscheduled.
- Pay records keep history: a pay change adds a new row with its effective date instead of overwriting the old one.
- Deactivating a staff member keeps all of their records.
- Leave policy is a clinic-wide setting: a yearly allowance (15 days by default), which leave types count against it, an optional separate sick allowance, the leave year (calendar year by default), a carry-over cap, and pro-rating for new hires. Leave balance (the allowance minus approved leave of the counted types) and credential status (valid, expiring soon, expired) are calculated, not stored.
- Open: how many leave days does each person get per year? The leave balance story needs this rule. Answer: Per person 15 days per year.

Decided: which leave types count against the yearly allowance is a setting, not a fixed rule. The clinic can use one shared bank or give sick leave its own allowance, depending on its state's rules.

Rules added after team review

- A coverage rule sets the minimum headcount for one job title at one location on one weekday. The check uses shifts and approved leave, skips closed days, and only warns.
- A credential type can be required for job titles. If a required credential is expired on a shift date, publishing is blocked unless the Owner / Admin overrides it, and the override is logged.
- Leave is counted only on days the person would have worked, in half-day steps, and closed days do not count against the balance.
- Unpaid break minutes are deducted from attendance by default. Short paid breaks are never deducted, and a manager can correct a break with a logged change.
- Weekly hours add up across both locations, using the workweek start and threshold from the clinic settings. Only overtime-eligible staff trigger the warning, and overtime pay is not calculated.
- The Owner / Admin's own leave is recorded as approved, or approved by a second Owner / Admin if there is one.

Part 6: Design Kit and Design Rules

The AI builds most screens, so Part 6 is a small design kit that tells it exactly how the app looks and behaves: tokens, one component base, five hero screens you design, and strict rules. Every other screen

copies the closest hero screen, so quality and consistency come from your examples, not from the AI's defaults.

The design kit

Item	What it is	Who makes it
Design tokens	Colors, type scale, spacing, radius, and status colors, as a JSON or CSS variables file	You
Component base	shadcn/ui on Tailwind, restyled with your tokens so no default look survives	The AI sets it up in milestone 1, you approve it
Hero screens	The five screens below, designed in Figma on desktop and phone as needed	You
Design rules	The rules below, saved as design/design-rules.md	Drafted here, you edit
Content list	Button labels, error messages, empty-state text, and notification wording	You and the AI
Screen inventory	Every screen with the roles that see it	Starter list in the Part 6 checklist workbook

Design direction to lock first

A recognizable look comes from decisions made once and applied everywhere. Fill in the last column before designing the hero screens.

Decision	What to settle	Your choice
Personality	Three words that describe the tone, for example calm, precise, human	
Accent color	One hue used only for primary actions and the current selection, plus a neutral palette	
Typefaces	One for headings and one for body and data, or one family in two weights. Pick them on purpose instead of accepting the default	

Decision	What to settle	Your choice
Shape	Radius values and whether areas are separated by borders or by background tints	
Density	Comfortable for staff screens, compact for manager tables	
Signature elements	The shift card, the status badge, and the page header, designed once and reused everywhere	

The five hero screens

Hero screen	Device	Who uses it	Must show
Weekly schedule	Desktop	Practice Manager, Owner / Admin	Shifts per person per day for both locations, add and edit, conflict warning, weekly hours per person with an overtime warning, a staffing coverage row, the expired-licence block, publish
My schedule	Phone	All roles	Upcoming shifts at either location, with the location on every shift
Clock in and out	Phone	All roles	One large button, current status, today's shift, the location selector, the missing clock-out message
Leave request and inbox	Phone and desktop	All roles request, managers decide	Dates, type, status, balance, and the manager inbox with team calendar overlap
Practice Manager dashboard	Desktop	Practice Manager	Who is working today, pending leave requests, expiring credentials, missed clock-outs

Design rules

Save this as design/design-rules.md. The rules file in Part 9 points to it, so the AI reads it on every task. The rules work in four ways: they remove the AI's defaults, fix every value to the tokens, define each repeated pattern once, and make the AI check its own work.

```
# Design rules

## Source of truth
- design/tokens.json is the only source for color, type, spacing, radius, and shadow values. Never write a raw hex color, pixel size, or font name in a component.
- The hero screens in design/hero-screens/ are the visual reference. Before building any screen, find the closest hero screen and copy its layout, spacing, and component use.
- If a component or pattern you need does not exist, stop and ask. Do not invent one.

## Anti-generic rules
- Never ship a component library's default theme. Every component is restyled with the tokens.
- No gradients, glassmorphism, glows, stacked drop shadows, decorative shapes, or stock illustrations.
- No emoji as icons. Use one icon set, one stroke weight, one size scale.
- No marketing-style layouts: no centered hero sections, oversized headings, or grids of equal cards where a table or list fits the data better.
- Use one accent color only. It marks primary actions and the current selection, and nothing else.
- Use the three signature elements (shift card, status badge, page header) exactly as designed. Never restyle them per screen.
- Do not add decoration to fill space. Empty space is fine.

## Consistency
- Spacing comes only from the scale: 4, 8, 12, 16, 24, 32, 48. Page padding and the gaps between sections use the same values on every screen.
- Type comes only from the token scale: page title, section title, body, small, label. Headings never skip levels.
- Radius has three values: controls, cards, badges. No others.
- Prefer 1px borders or background tints to separate areas. Shadows are only for menus, popovers, and dialogs.
- Every list of records uses the same table component: same row height, header style, sorting, and empty state.
- Every form uses the same layout: label above the field, helper text below, error text below in the error color, primary button at the bottom right on desktop and full width on phones.
- One primary button per area of a screen. Destructive actions use the destructive style and ask for confirmation.
- Status always uses the badge component with text and color, never color alone: pending, approved, rejected, flagged, expiring, expired, active, deactivated.
- Dates and times use one format everywhere, in the clinic's time zone, for example Mon, Oct 12 and 9:00 AM.
- People are shown as First Last. Roles and locations use the exact names from the content list.

## States
- Every screen and every data component has loading, empty, error, and no-permission states.
```

```
- Loading uses skeletons that match the final layout. Empty states say what the
screen is for and offer the next action. Errors say what happened and how to fix it.
- The shift conflict, minimum staffing, overtime, and missing clock-out warnings use
the warning pattern from the hero screens, and the expired-licence block uses a
blocking variant of it. Never invent a new alert style.

## Roles and data
- Hide what a role cannot use. Do not show disabled buttons for actions a role never
has.
- Pay data appears only in the pay section of a profile for owner_admin, and in each
person's own pay view.
- Always show the location on shifts and on any list that spans both locations.

## Phone and desktop
- Staff screens (my schedule, clock in and out, leave request, notifications) are
designed for a 360px phone first, then widened.
- Manager screens (weekly schedule, staff directory, reports) are designed for
desktop first and must work on a tablet. On a phone they switch to a day view or a
list instead of a squeezed grid.
- Tap targets are at least 44 by 44 px. The clock in and out button is the largest
control on its screen.
- The page never scrolls sideways. Wide tables scroll inside their own container.

## Accessibility
- Text contrast is at least 4.5 to 1, and interface elements at least 3 to 1.
- Everything works with the keyboard, with a visible focus ring from the tokens.
- Every input has a visible label. Icon-only buttons have an accessible name.
- Never rely on color alone for meaning.
- Motion is only for feedback, 150 to 200 ms, and respects the reduced motion
setting.

## Content
- Use the wording in design/content-list.md. Buttons start with a verb, such as
Request leave. Use sentence case everywhere.
- Keep the tone plain and calm: no exclamation marks, no jokes.
- Error messages never blame the user and always say what to do next.

## Check your work
- After building a screen, name the hero screen it copies and any new pattern it
introduces.
- Check every screen at 360, 768, and 1280 px wide before calling it done.
- Before finishing a task, review the screen against these rules and list any rule
you could not follow.
- Do not change the styling of screens that are not part of the task.
```

How to use the kit during the build

1. Put the tokens, exported hero screens, design rules, and content list in the design folder before milestone 1.
2. At the start of each milestone, tell the AI which hero screens and rules apply.
3. Review every new screen in the browser at phone and desktop sizes, and compare it with the closest hero screen.
4. When you correct the same thing twice, add a rule to design/design-rules.md.

5. When a screen needs a pattern the hero screens do not cover, design it once in Figma and add it to the kit before the AI builds more screens with it.

Part 7: Tech Stack and Architecture Notes

One Next.js app with a Supabase (Postgres) backend covers everything in version 1. This is a proposal built on popular, well-documented tools, which AI coding tools handle best. Confirm it before building.

Layer	Proposed choice	Why
Frontend	Next.js (React) with TypeScript	One codebase for the manager desktop views and the staff mobile views
Styling	Tailwind CSS and shadcn/ui components, restyled with the design tokens from Part 6	Your Figma tokens map directly to code
Database	Supabase (Postgres)	The data model in Part 5 is relational, with clear links between records
Authentication	Supabase Auth with email invitations	Covers the invite and set-password flow
Access control	Postgres row-level security by role and location	Enforces the Part 3 matrix in the database, so a screen bug cannot expose pay data
Notifications	In-app list plus email through a transactional email service	Schedule published, leave decisions, and expiry reminders
Scheduled jobs	One daily job for credential expiry reminders	Sends the 30-day reminders without anyone remembering to
Hosting	Vercel for the app, Supabase for the database	Simple deployment with automatic backups available
Testing	Automated tests for permissions, the four schedule checks, and the Must flows	These are the riskiest parts of the app

Architecture notes

- Permissions live in the database. The screens hide what a role should not see, but row-level security is what actually blocks access.
- Pay records sit in their own table with their own access rule, so general staff queries can never return them.

- The shift conflict check runs on the server and looks across both locations, so every screen gets the same result.
- Clock-in and clock-out times come from the server's clock, not the phone's, so they cannot be edited from a device.
- The audit log is written by the database whenever schedules, leave decisions, attendance, pay, or roles change, so no code path can skip it.
- Personal data and pay data use HTTPS, encrypted storage, and daily backups.
- Version 1 is a responsive website. Native mobile apps stay out of scope.
- Open: do both locations share one time zone? Shift and clock times need a stated zone. Answer: Yes, one time zone
- Open: which country is the clinic in? It decides the hosting region and the privacy rules for staff and pay data. Answer: USA

What the answers change

- Host the app and the database in a US region.
- The clinic has one time zone, kept as a single app-wide setting. Clock-in and clock-out times are stored as exact timestamps and shown in that zone. Shift times are stored as a local date and time in that zone.
- Keep staff, attendance, and pay records instead of deleting them. US employers generally have to keep payroll and time records for a set period (federal rules mention two to three years, and state rules can be longer), so confirm the exact period with the clinic's accountant or lawyer.
- Do not store Social Security numbers, bank account details, or patient information in version 1. They carry heavy legal duties, and the app does not need them.
- The app holds no patient information, so patient privacy rules such as HIPAA are not the main concern, as long as that stays true. Staff and pay data still fall under US federal and state employment and privacy law, so have a lawyer check before launch.

Checks and settings from the team review

- The server runs four schedule checks: overlap or leave conflict, minimum staffing, expired required licence, and overtime. The licence check blocks publishing unless the Owner / Admin overrides it. The other three only warn.
- Policies live in settings tables (leave policy, clinic setting, coverage rule, credential type, closed day), not in code. Every change to them is written to the audit log.
- Weekly hours are calculated on the server from scheduled and actual times, added up across both locations for the workweek set in the clinic settings.
- Break deductions and overtime flags are calculated from the stored times when a report is read, so a correction updates every report automatically.

Part 8: Milestone Plan

The app is built in ten milestones, one at a time. Each ends with a "done when" test, and the next one starts only after that test passes.

#	Milestone	What gets built	Done when
1	Project setup	Repository, Next.js app, database and hosting in a US region, design tokens and component base from Part 6, rules file	A blank app deploys and connects to the database
2	Sign in, roles, and locations	Email invitations, set password, sign in, four roles, home location, row-level security	An invited test user signs in and sees only what their role and location allow
3	Staff profiles	Staff directory, profile, add, edit, deactivate, location assignment, job title, overtime-eligible status	Owner / Admin and Practice Manager can manage staff within their scope, and other roles cannot
4	Settings and policies	Job titles, clinic settings (time zone, workweek start, overtime threshold, default break), leave policy, closed-day calendar, coverage rules, credential types	The Owner / Admin can change every setting and the audit log records it, and no other role can
5	Credentials	Credentials per person linked to credential types, expiring-soon list, expired status	A required credential shows as valid, expiring soon, or expired on the right dates
6	Shift scheduling and schedule checks	Weekly calendar, add, edit, and delete shifts at either location, weekly hours per person, the four checks (overlap or leave, minimum staffing, expired required licence, overtime), licence block with logged Owner / Admin override, publish, my	A manager builds and publishes a week, every check fires on test data, a licence block stops publishing until overridden, and staff see the week on their phones

#	Milestone	What gets built	Done when
		schedule, publish notification	
7	Leave management	Request leave with half days, approve or reject with a comment, Owner / Admin leave recorded automatically, request status, team leave calendar with staffing impact, leave balance, notifications	A request goes from submitted to decided, balances follow the leave policy with closed days not deducted, and approved leave appears on the calendar and in schedule checks
8	Attendance and clock in and out	Clock in and out with a location choice, unscheduled clock-in flag, automatic unpaid break deduction with logged correction, flagged missing clock-outs, overtime flag, my attendance, attendance records	A full shift is recorded with breaks deducted, an unscheduled clock-in and a missing clock-out are both flagged, corrected, and logged, and weekly hours add up across both locations
9	Pay and compensation	Pay records with history, visible only to the Owner / Admin and to each person for their own	A Practice Manager, clinical staff member, and receptionist cannot see anyone else's pay by any route
10	Reports, reminders, audit log, and launch checks	Hours reports (scheduled, worked, unpaid breaks, paid hours, overtime) and leave reports, 30-day credential reminders, audit log view, accessibility and security review, backups, final deploy	A real clinic manager completes every Must flow without help

Working rules for every milestone

1. Give the AI the rules file, the relevant parts of this document, and the design for that milestone's screens.
2. Ask it for a short plan before it writes code.
3. Test the result as each of the four roles, at both locations.
4. Save a Git commit when the done-when test passes.
5. Start a fresh AI chat for the next milestone instead of continuing a long one.

Part 9: Rules File for the AI

The rules file is a short document at the root of the project, often named CLAUDE.md, that the AI reads at the start of every session. Below is a first version to copy in. Adjust the file names after Part 6, and update it whenever a decision changes.

```
# Dental Clinic Staff Management App

## What this is
A web app for a dental clinic in the USA with two locations. It manages staff
profiles, shift schedules, leave (yearly allowance of 15 days by default, set in the
leave policy setting), attendance (clock in and out), credentials, and pay records.
Version 1 includes only the features marked Must in docs/feature-list.md.

## Stack
- Next.js with TypeScript
- Tailwind CSS, using the design tokens in design/tokens.json
- Supabase: Postgres, Auth, and row-level security, in a US region
- App hosted on Vercel

## Folder structure
- app/ for routes and pages
- components/ for shared UI, built on shadcn/ui and restyled with the tokens
- lib/ for database access, permission helpers, and date helpers
- supabase/migrations/ for database changes, one file per change
- design/ for tokens, design rules, hero screens, and the content list
- docs/ for the product brief, feature list, roles matrix, flows, data model, and
milestones

## Naming
- Use the data model names: Staff, Location, Shift, LeaveRequest, AttendanceRecord,
Credential, PayRecord, Notification, AuditLogEntry, Invitation, LeavePolicy,
CredentialType, ClinicSetting, CoverageRule, ClosedDay, JobTitle
- Components in PascalCase, files in kebab-case, database tables in snake_case

## Roles
- Four roles: owner_admin, practice_manager, clinical_staff, reception
- Permissions follow docs/roles-matrix.md exactly
- Each person has one home location. Each shift has its own location, which can
differ.

## Always
- Enforce permissions in the database with row-level security, not only in the UI
- Check every new screen as each of the four roles, at both locations
```

```
- Take clock-in and clock-out times from the server's clock
- Show and enter all times in the clinic's single time zone setting
- Read every policy value from the settings tables (leave policy, coverage rules,
credential requirements, overtime threshold, workweek start, default break, closed
days), never hardcode them
- Run the schedule checks on the server, across both locations: overlap or leave
conflict, minimum staffing, expired required licence, and overtime
- Add up weekly hours across both locations, for the workweek set in the clinic
settings
- Write every schedule change, leave decision, attendance correction, pay edit, and
role change to the audit log
- Use the wording in design/content-list.md for labels and messages
- Follow design/design-rules.md on every screen, and copy the layout of the closest
hero screen
- Use only values from design/tokens.json for color, type, spacing, radius, and
shadow
- Make every screen work on a phone
- Add a database migration for every structure change
- After each task, say what changed and how to test it

## Never
- Never return pay data except to owner_admin or to the person it belongs to
- Never let a manager approve their own leave or correct their own attendance
- Never store Social Security numbers, bank account details, or patient information
- Never put keys or secrets in the code; use environment variables
- Never build features marked Should or Later unless asked
- Never change the database structure without telling me first
- Never delete staff records; deactivate them instead
- Never let a schedule with an expired required licence be published without a logged
Owner / Admin override
- Never discard unscheduled clock-in time; record it and flag it
- Never deduct short paid breaks
- Never use a component library's default styling, gradients, or any pattern that is
not in the design kit

## Working style
- Work on one milestone at a time from docs/milestones.md
- Ask questions before writing code if anything in the docs is unclear
- Keep changes small, and stop for a Git commit after each working step
```

How to use it

- Save the file as CLAUDE.md in the project root. Put Parts 1 to 8 in the docs folder as separate files, using the names above.
- Keep it short. Long rules files get ignored in practice, so move detail into the docs and point to it.
- When the AI makes the same mistake twice, add a rule for it.

Decisions to confirm with the clinic

The rules above use sensible defaults, but the clinic has to confirm the values before milestone 4. Some of these depend on state law, so the clinic's accountant or employment lawyer should check them.

- The minimum headcount per location, job title, and weekday.

- Which credentials each job title requires, which ones block scheduling when expired, and any state registration or certification a dental assistant needs for specific duties.
- Whether the Owner / Admin role is one person or several.
- The leave policy values: whether the 15 days are granted upfront or accrued, the leave year, the carry-over cap, pro-rating for new hires, rules for part-time staff, and which leave types count against the allowance.
- The closed days for each location, if they differ.
- Which staff are overtime-eligible, and the workweek start day.
- The default unpaid break length, and whether any state meal or rest break rules apply.
- Whether staff should also clock out and in for breaks (currently Later).
- Which state each location is in. If they are in different states, sick leave, break, and overtime rules may differ per location.
- Whether clocking in from home is a concern. If it is, add a location QR code or a clinic-network check.

Next steps

Parts 6 to 9 are drafted. What is left is your design work for Part 6, then the build.

- **Part 6 design work:** fill in the design direction table, design the tokens and the five hero screens in Figma, and write the content list.
- **Then:** save Parts 1 to 9 as separate files in the project's docs folder, put the design files in the design folder, and start milestone 1.